TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



LÊ PHÚ – 524H0121
VŨ GIANG HUY – 524H0095

HỆ THỐNG THANH TOÁN HỌC PHÍ IBANKING SỬ DỤNG KIẾN TRÚC MICROSERVICES

BÁO CÁO GIỮA KỲ
KIẾN TRÚC HƯỚNG DỊCH VỤ

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



LÊ PHÚ – 524H0121
VŨ GIANG HUY – 524H0095

HỆ THỐNG THANH TOÁN HỌC PHÍ IBANKING SỬ DỤNG KIẾN TRÚC MICROSERVICES

BÁO CÁO GIỮA KỲ
KIẾN TRÚC HƯỚNG DỊCH VỤ

Giảng viên hướng dẫn
Dương Hữu Phúc

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

Chúng em xin chân thành cảm ơn giảng viên hướng dẫn – thầy **Dương Hữu Phúc** đã tận tình hướng dẫn và hỗ trợ chúng em trong suốt quá trình thực hiện đồ án.

Chúng em cũng xin gửi lời cảm ơn đến Khoa Công nghệ thông tin và Trường Đại học Tôn Đức Thắng đã tạo môi trường và điều kiện thuận lợi để chúng em hoàn thành tốt đề tài này.

Chúng em xin chân thành cảm ơn!

TP. Hồ Chí Minh, ngày 24 tháng 10 năm 2026

Tác giả

(Ký tên và ghi rõ họ tên)

Huy

Phú

Vũ Giang Huy – Lê Phú

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH
TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Chúng tôi xin cam đoan đây là công trình nghiên cứu của nhóm và được sự hướng dẫn khoa học của thầy Dương Hữu Phúc. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa từng công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được các tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong Dự án còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào chúng tôi xin hoàn toàn chịu trách nhiệm về nội dung Đề tài của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do chúng tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày 24 tháng 10 năm 2026

Tác giả

(Ký tên và ghi rõ họ tên)

Huy

Phú

Vũ Giang Huy – Lê Phú

MỤC LỤC

CHƯƠNG 1. MỞ ĐẦU VÀ TỔNG QUAN ĐỀ TÀI 1
1.1 Lý do chọn đề tài 1
1.2 Mục tiêu thực hiện đề tài 1
1.3 Scope of Project 1
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 2
2.1 Microservices Architecture & API Gateway 2
2.2 Distributed Transaction 2
2.3 Concurrency Control 2
2.4 Asynchronous Communication 2
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 4
3.1 Use Case Diagram 4
3.2 Architecture Diagram 4
3.3 ERD Diagram 5
3.4 Workflow Diagram 6
3.5 Sequence Diagram 7
CHƯƠNG 4. CÀI ĐẶT VÀ TRIỂN KHAI 9
4.1 Technologies Used 9
4.2 Data Consistency & Concurrency Control 9
CHƯƠNG 5. KẾT QUẢ 10
5.1 User Interface 10
5.2 Concurrency Testing 13
5.3 Saga Rollback & Compensation 14
5.4 Technical Deliverables Evaluation 15
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 17
6.1 Kết luận 17
6.2 Hướng phát triển 17
TÀI LIỆU THAM KHẢO 18

CHƯƠNG 1. MỞ ĐẦU VÀ TỔNG QUAN ĐỀ TÀI

1.1 Lý do chọn đề tài

Trong các đợt cao điểm thu học phí, các hệ thống thông tin trường đại học truyền thống thường đối mặt với nguy cơ quá tải, nghẽn mạng và xuất hiện xung đột dữ liệu như trừ tiền trùng lặp hoặc nhiều giao dịch cùng gạch nợ cho một sinh viên. Để khắc phục triệt để các hạn chế này, đề tài xây dựng hệ thống thanh toán học phí iBanking phân tán trên nền tảng kiến trúc Microservices, nhằm nâng cao khả năng chịu tải và bảo đảm tính nhất quán của dữ liệu tài chính.

Đề tài tập trung giải quyết các bài toán kỹ thuật trọng tâm: phân tách dịch vụ độc lập theo nguyên tắc Database-per-Service, điều phối chuỗi giao dịch phân tán bằng Saga Pattern, ngăn chặn race condition bằng khóa bi quan (Pessimistic Locking), và tích hợp luồng gửi mã OTP qua email thực tế phục vụ xác thực giao dịch.

1.2 Mục tiêu thực hiện đề tài

• Xây dựng hệ thống thanh toán học phí trực tuyến mô phỏng giao dịch ngân hàng điện tử hiện đại.

• Bảo đảm tính nhất quán dữ liệu trong môi trường phân tán thông qua mô hình Saga Orchestration.

• Xử lý triệt để bài toán tương tranh: một tài khoản gửi yêu cầu thanh toán liên tục (Double-spending) hoặc nhiều người cùng thanh toán cho một khoản học phí.

• Tích hợp cơ chế xác thực hai lớp (2FA) sử dụng mã OTP gửi qua giao thức SMTP đến email người dùng.

1.3 Scope of Project

Phạm vi thực hiện của đề tài bao gồm 6 dịch vụ backend độc lập (Auth Service, User Service, Tuition Service, Payment Service, OTP Service, Notification Service), 1 API Gateway tập trung quản lý định tuyến và xác thực JWT, cùng giao diện người dùng xây dựng bằng React.js hỗ trợ đầy đủ quy trình tra cứu, thanh toán và xuất biên lai điện tử.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Microservices Architecture & API Gateway

Kiến trúc Microservices phân rã ứng dụng nguyên khối thành các dịch vụ độc lập xoay quanh từng nghiệp vụ cụ thể. Mỗi dịch vụ sở hữu cơ sở dữ liệu riêng biệt (Database-per-Service), đảm bảo tính đóng gói dữ liệu và giảm thiểu phụ thuộc lẫn nhau. API Gateway hoạt động như điểm chạm duy nhất cho các yêu cầu từ phía máy khách, đảm nhiệm định tuyến, xác thực token JWT, giới hạn tần suất yêu cầu và bảo vệ cấu trúc nội bộ của các dịch vụ backend.

2.2 Distributed Transaction

Trong môi trường Database-per-Service, giao dịch kinh doanh trải rộng trên nhiều cơ sở dữ liệu không thể sử dụng cơ chế khóa phân tán Two-Phase Commit (2PC) do nguy cơ nghẽn cổ chai và giảm tính sẵn sàng. Hệ thống áp dụng Saga Pattern theo mô hình điều phối tập trung (Orchestration): Payment Service đóng vai trò Orchestrator gửi tuần tự các lệnh giao dịch cục bộ. Khi có bước thất bại (ví dụ: lỗi thanh toán học phí hoặc hết hạn OTP), Orchestrator sẽ kích hoạt các hành động bù trừ (Compensating Transactions) để hoàn tiền về tài khoản, bảo đảm tính nhất quán cuối cùng (Eventual Consistency).

2.3 Concurrency Control

Hiện tượng Race Condition xuất hiện khi nhiều luồng hoặc yêu cầu cùng lúc truy xuất và chỉnh sửa một tài nguyên dùng chung, dẫn tới nguy cơ trừ tiền âm tài khoản hoặc gạch nợ học phí nhiều lần. Hệ thống sử dụng cơ chế khóa bi quan (Pessimistic Locking) với câu lệnh SELECT ... FOR UPDATE tại User Service và Tuition Service. Khi một giao dịch đang đọc và cập nhật dữ liệu, bản ghi sẽ bị khóa độc quyền; các yêu cầu đồng thời đến sau phải xếp hàng chờ giao dịch đầu hoàn tất, ngăn chặn hoàn toàn việc dữ liệu bị ghi đè sai lệch.

2.4 Asynchronous Communication

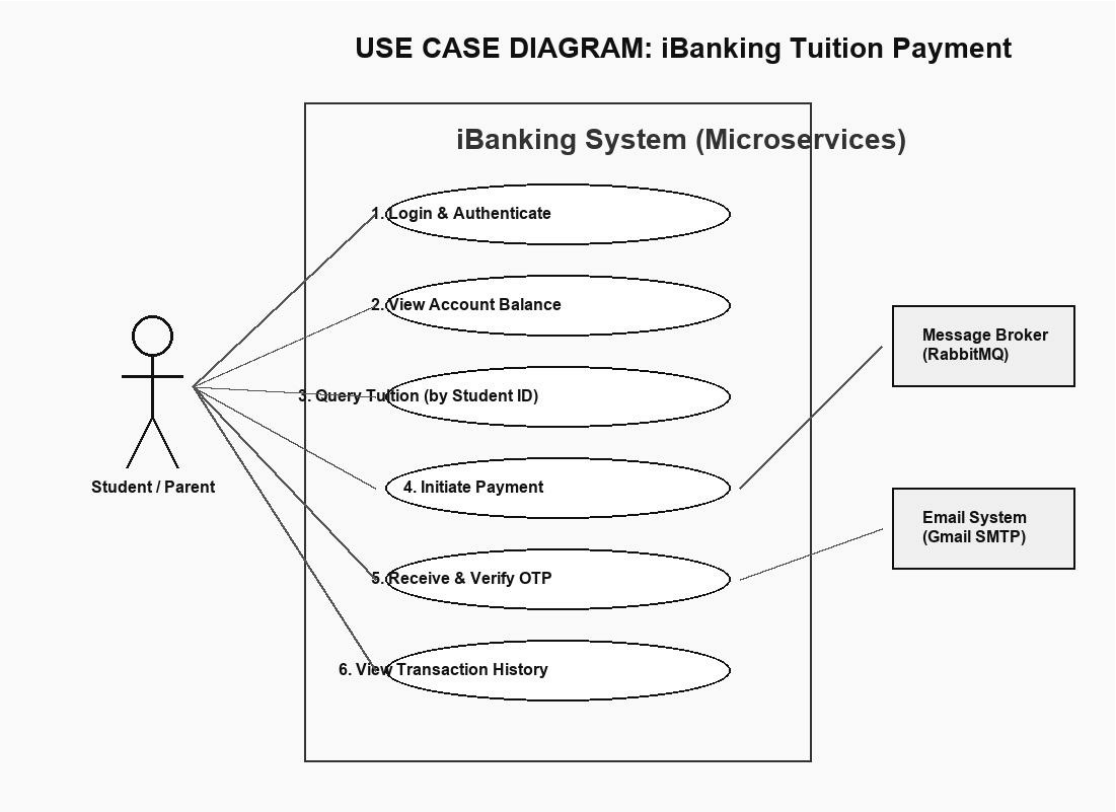
Hệ thống sử dụng hàng đợi thông điệp RabbitMQ để triển khai giao tiếp bất đồng bộ giữa các Microservices. Cơ chế này được áp dụng cho các tác vụ cần thời gian xử lý I/O mạng như gửi email mã OTP và gửi thư xác nhận thanh toán thành công qua Notification Worker. Việc giao tiếp bất đồng bộ giải phóng luồng xử lý chính

của Payment Service, giảm độ trễ phản hồi cho người dùng và bảo đảm khả năng chịu lỗi khi dịch vụ email bên thứ ba gặp sự cố tạm thời.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Use Case Diagram

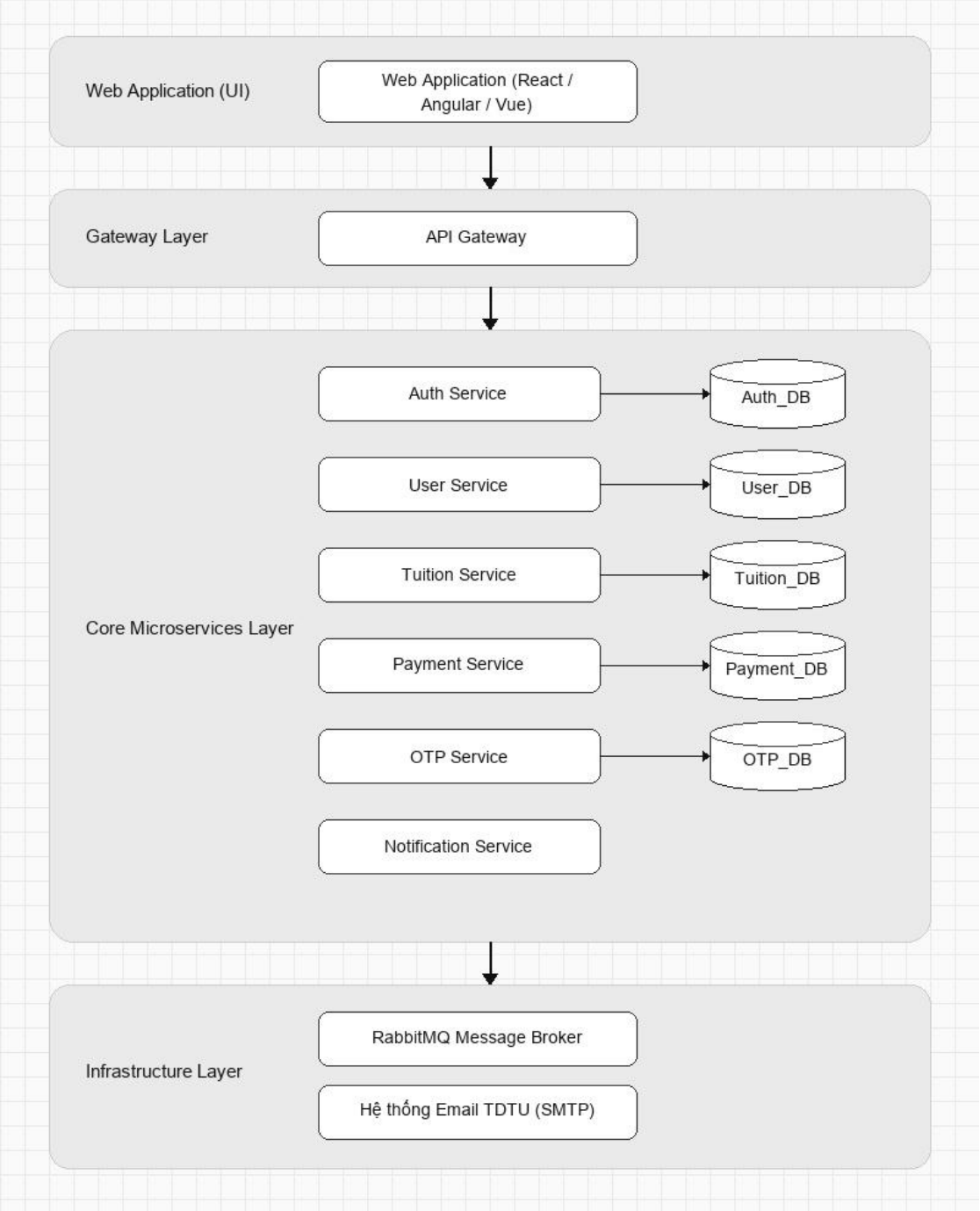
Sơ đồ Use Case thể hiện các ca sử dụng chính của hệ thống với hai tác nhân (Actor) chính là Sinh viên và Phụ huynh. Người dùng thực hiện đăng nhập vào hệ thống iBanking qua API Gateway, tra cứu công nợ học phí theo mã số sinh viên (MSSV), xác nhận chi phí, nhận và xác thực mã OTP qua email, cũng như xem và in biên lai điện tử sau khi hoàn tất giao dịch.



Hình 3.1: Use Case Diagram hệ thống iBanking TDTU

3.2 Architecture Diagram

Kiến trúc hệ thống được phân tầng rõ ràng theo mô hình Microservices và Database-per-Service. Mọi yêu cầu từ ứng dụng Frontend (React.js) đều gửi qua API Gateway làm cổng giao tiếp duy nhất. Hệ thống backend bao gồm 6 dịch vụ độc lập: Auth Service, User Service, Tuition Service, Payment Service, OTP Service và Notification Service. Các dịch vụ duy trì cơ sở dữ liệu PostgreSQL độc lập và giao tiếp bất đồng bộ qua RabbitMQ Message Broker.

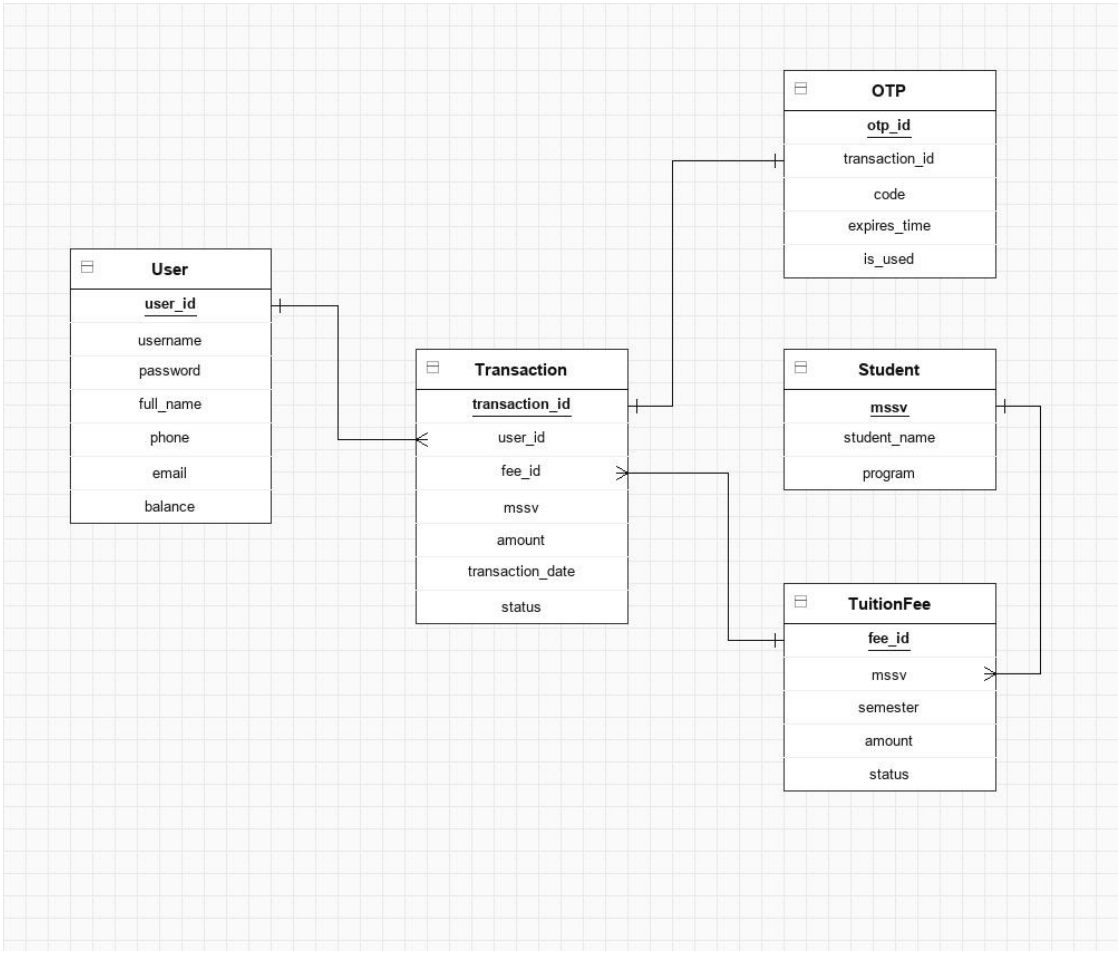


Hình 3.2: Architecture Diagram hệ thống iBanking TDTU

3.3 ERD Diagram

Sơ đồ thực thể liên kết (ERD) thể hiện cấu trúc bảng của từng phân hệ dịch vụ độc lập. Nhằm tuân thủ nguyên tắc cô lập dữ liệu, giữa các cơ sở dữ liệu không thiết lập ràng buộc khóa ngoại vật lý chéo, mà liên kết thông qua các định danh logic (như user_id, mssv, tuition_fee_id). Cấu trúc bảng gồm: users và accounts (User

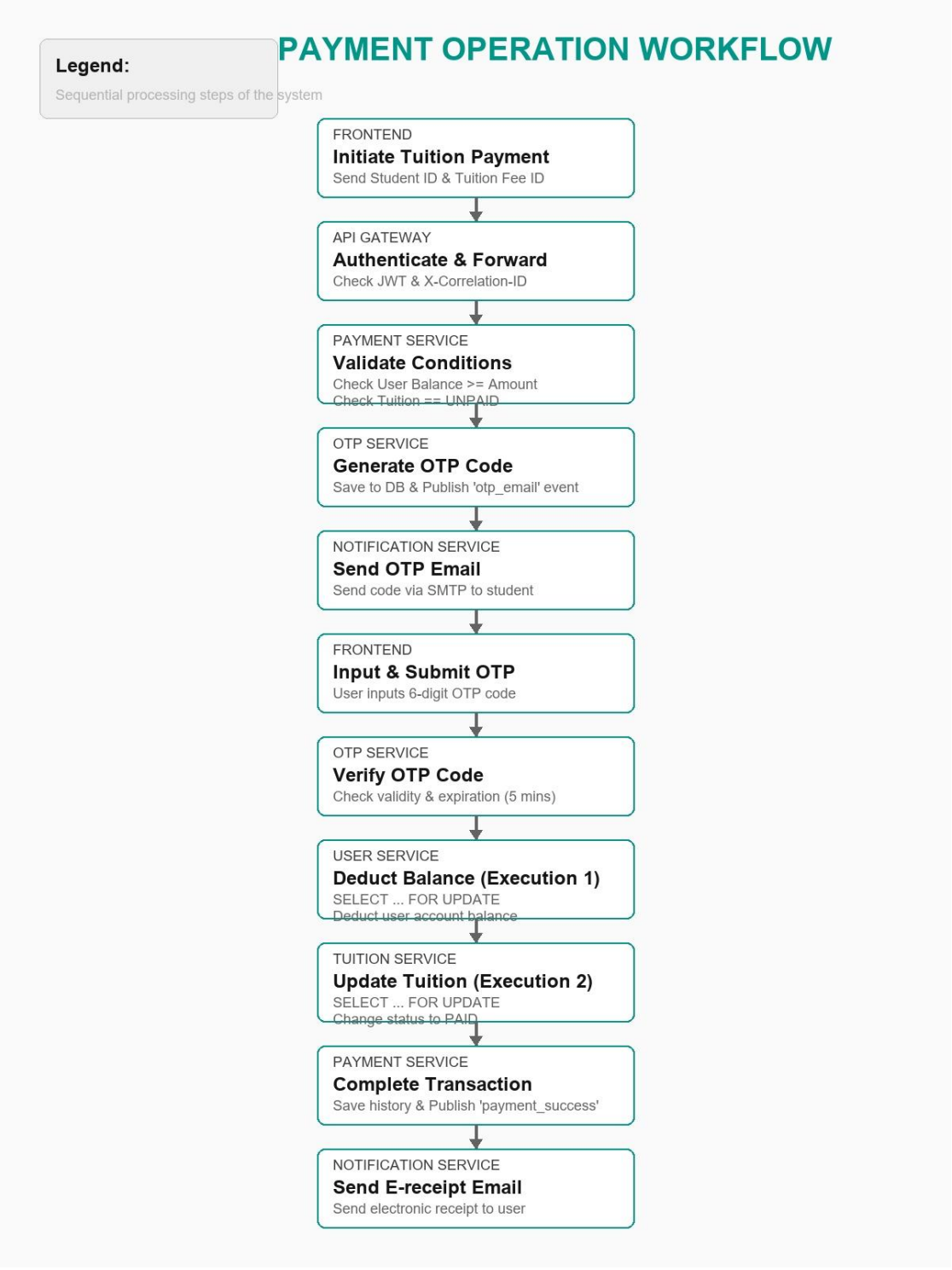
Service), students và tuition_fees (Tuition Service), transactions và payment_audit_logs (Payment Service), và otp_requests (OTP Service).



Hình 3.3: ERD Diagram cơ sở dữ liệu các phân hệ

3.4 Workflow Diagram

Sơ đồ luồng hoạt động mô tả tiến trình thanh toán học phí qua 4 giai đoạn tuần tự: (1) Tra cứu học phí theo MSSV từ Tuition Service; (2) Đối soát số dư khả dụng và khởi tạo giao dịch tại Payment Service; (3) Sinh mã xác thực OTP, gửi qua hàng đợi RabbitMQ đến Notification Worker và nhập mã OTP xác thực; (4) Khóa dữ liệu bằng Pessimistic Locking, thực hiện trừ tiền tài khoản, gạch nợ học phí và xuất biên lai thanh toán điện tử.

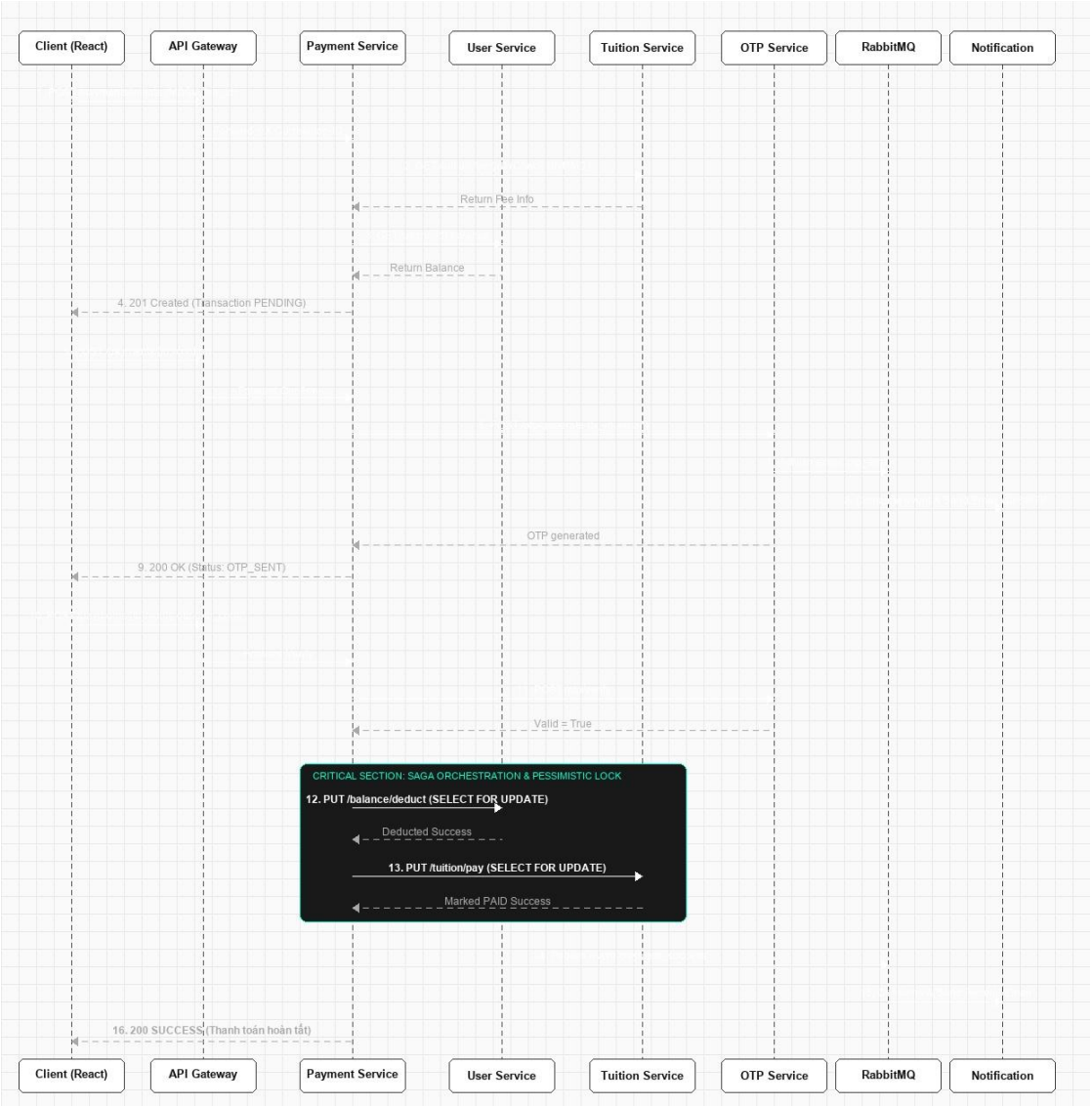


Hình 3.4: Workflow Diagram quy trình thanh toán học phí

3.5 Sequence Diagram

Sơ đồ tuần tự thể hiện chi tiết các bước tương tác giữa Frontend, API Gateway và các Microservices trong quy trình điều phối Saga. Payment Orchestrator đóng vai

trò trung tâm điều phối chuỗi gọi API sang User Service (trừ tiền), OTP Service (xác thực OTP), và Tuition Service (gạch nợ). Nếu xuất hiện lỗi tại bất kỳ bước nào (như học phí đã được thanh toán trước đó), Payment Orchestrator lập tức kích hoạt luồng bù trừ gửi yêu cầu hoàn tiền ngược về User Service để khôi phục số dư ban đầu.



Hình 3.5: Sequence Diagram luồng điều phối giao dịch Saga

CHƯƠNG 4. CÀI ĐẶT VÀ TRIỂN KHAI

4.1 Technologies Used

• Backend Services: Python (FastAPI), SQLAlchemy (Async I/O), PyJWT, Pydantic v2.

• Frontend Application: React.js, React Router v6, Axios, Plus Jakarta Sans typography.

• Database Storage: PostgreSQL 16 (5 cơ sở dữ liệu riêng biệt theo mô hình Database-per-Service).

• Message Broker: RabbitMQ (thư viện aio-pika) cho hàng đợi otp_email và payment_success.

• Email Notification: SMTP Gmail thông qua aiosmtplib phục vụ gửi thư xác thực thực tế.

• Containerization & Deployment: Docker, Docker Compose, kịch bản khởi chạy tự động 1-Click (start.bat cho Windows, start.command / start.sh cho macOS/Linux).

4.2 Data Consistency & Concurrency Control

• Chống thanh toán trùng tài khoản (Double-spending): Khi hai yêu cầu trừ tiền gửi đến đồng thời, User Service sử dụng lệnh SELECT ... FOR UPDATE để khóa độc quyền dòng dữ liệu tài khoản trong cơ sở dữ liệu. Yêu cầu thứ hai buộc phải đợi giao dịch đầu tiên hoàn tất rồi mới đọc số dư cập nhật, ngăn chặn tình trạng số dư bị trừ âm.

• Chống gạch nợ học phí trùng: Tuition Service áp dụng SELECT ... FOR UPDATE trên bản ghi học phí. Khi một giao dịch đã chuyển trạng thái sang PAID, giao dịch đồng thời thứ hai sẽ nhận phản hồi mã lỗi 409 Conflict. Payment Orchestrator bắt mã lỗi này và kích hoạt ngay hàm bù trừ (Refund) sang User Service.

• Idempotency Key: Header Idempotency-Key được tạo tự động từ giao diện người dùng kèm UUID v4 cho mỗi phiên thanh toán, giúp loại bỏ hoàn toàn các yêu cầu lặp lại do người dùng bấm đúp nút thanh toán.

CHƯƠNG 5. KẾT QUẢ

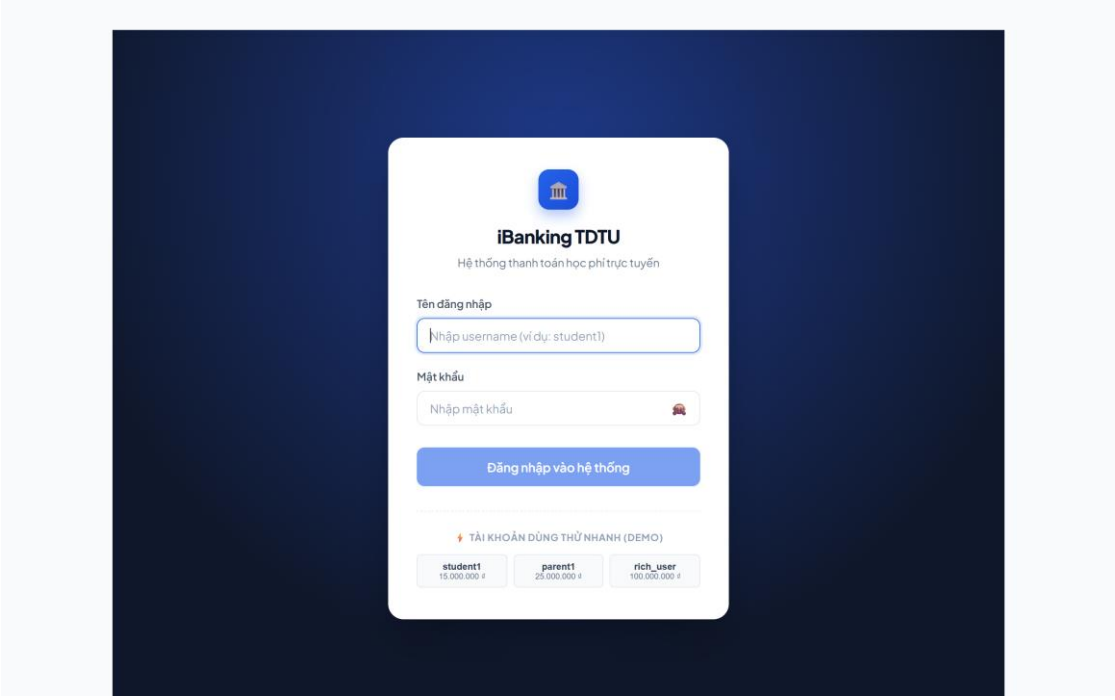
Hệ thống iBanking TDTU đã được triển khai hoàn chỉnh và kiểm thử thực tế. Nội dung chương này tổng hợp kết quả hoạt động của giao diện ứng dụng, dữ liệu kiểm thử tải tương tranh, cơ chế bù trừ giao dịch phân tán Saga, cùng bảng đối soát các yêu cầu kỹ thuật của đề tài.

5.1 User Interface

Ứng dụng web được xây dựng bằng React.js, tổ chức theo các chức năng chính của hệ thống ngân hàng trực tuyến:

5.1.1 Màn hình Đăng nhập

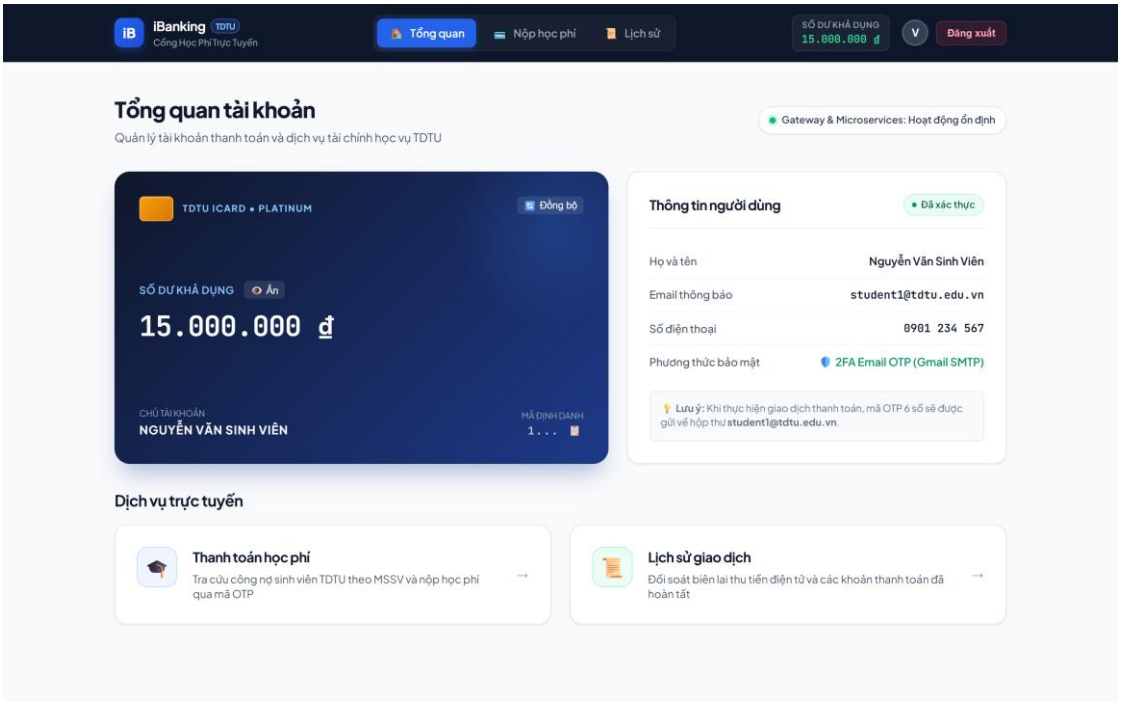
Màn hình đăng nhập chuyển tiếp thông tin xác thực qua API Gateway để nhận JWT token. Giao diện tích hợp các nút chọn nhanh tài khoản thử nghiệm (sinh viên, phụ huynh) nhằm hỗ trợ kiểm thử trực tiếp mà không cần nhập dữ liệu thủ công.



Hình 5.1: Màn hình đăng nhập hệ thống iBanking TDTU

5.1.2 Dashboard

Màn hình Dashboard hiển thị thông tin thẻ tài khoản, số dư khả dụng, trạng thái đồng bộ dữ liệu từ User Service và lối tắt dẫn trực tiếp đến chức năng nộp học phí.

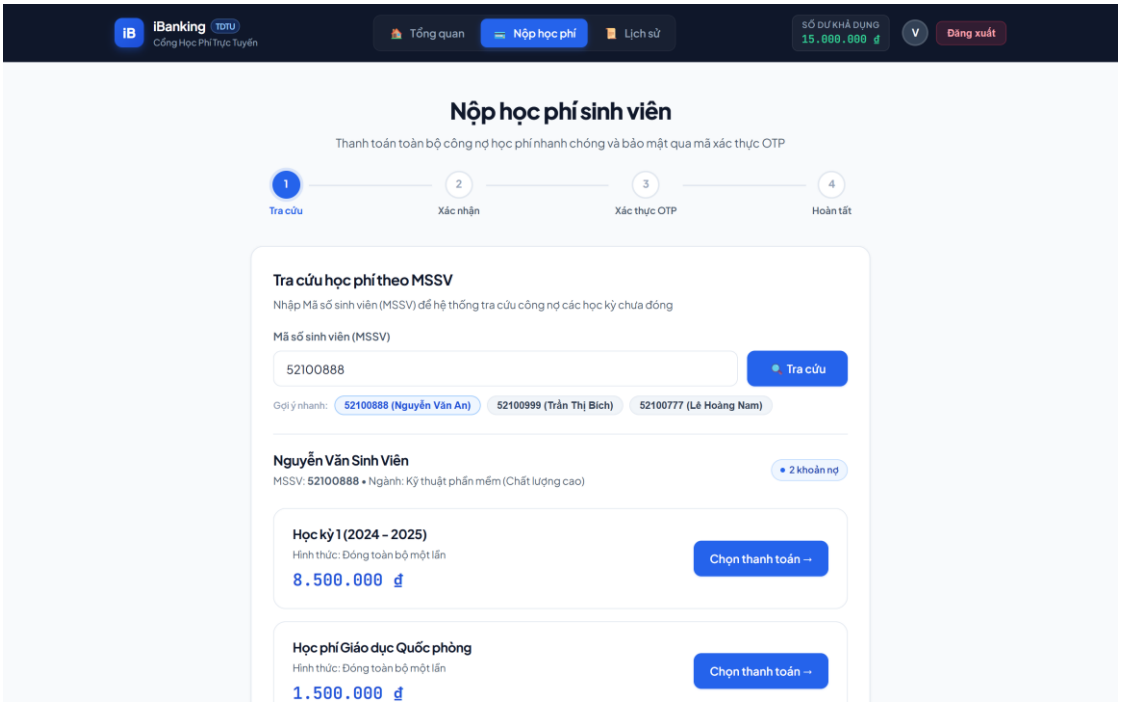


Hình 5.2: Bảng điều khiển tài khoản và số dư người dùng

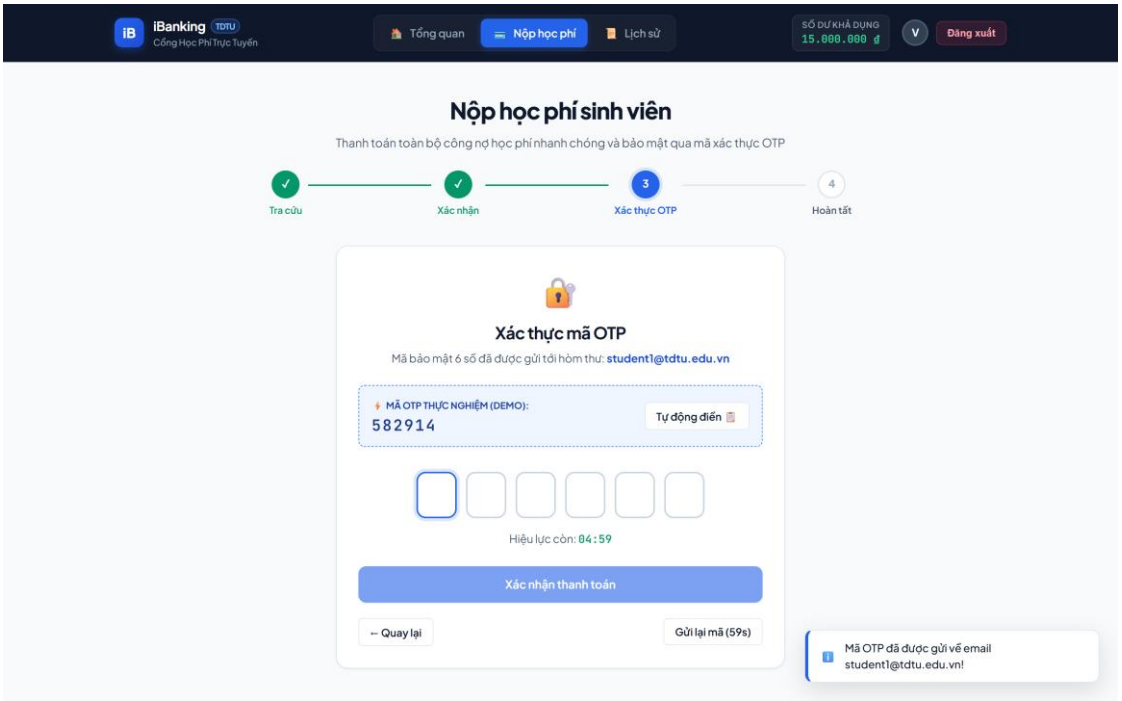
5.1.3 Quy trình nộp học phí

Chức năng nộp học phí được chia thành 4 bước tuần tự:

- Bước 1 (Tra cứu): Nhập mã số sinh viên (MSSV) để truy vấn danh sách các khoản học phí chưa thanh toán từ Tuition Service.
- Bước 2 (Xác nhận): Kiểm tra chi tiết khoản phí cần nộp và đối chiếu với số dư tài khoản của người dùng.
- Bước 3 (Xác thực OTP): Nhập mã OTP 6 chữ số gửi qua email, kèm bộ đếm thời gian hiệu lực 5 phút.
- Bước 4 (Biên lai điện tử): Hệ thống thông báo kết quả và hiển thị biên lai sau khi hoàn tất giao dịch.



Hình 5.3: Màn hình tra cứu và xác nhận thông tin học phí

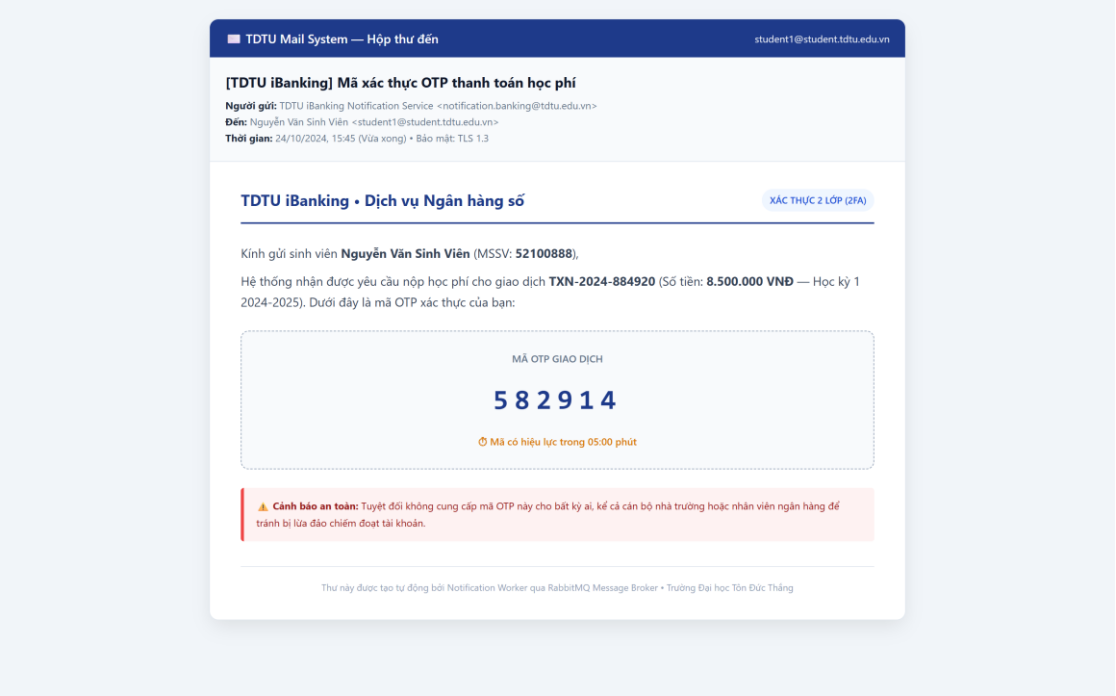


Hình 5.4: Màn hình xác thực giao dịch bằng mã OTP

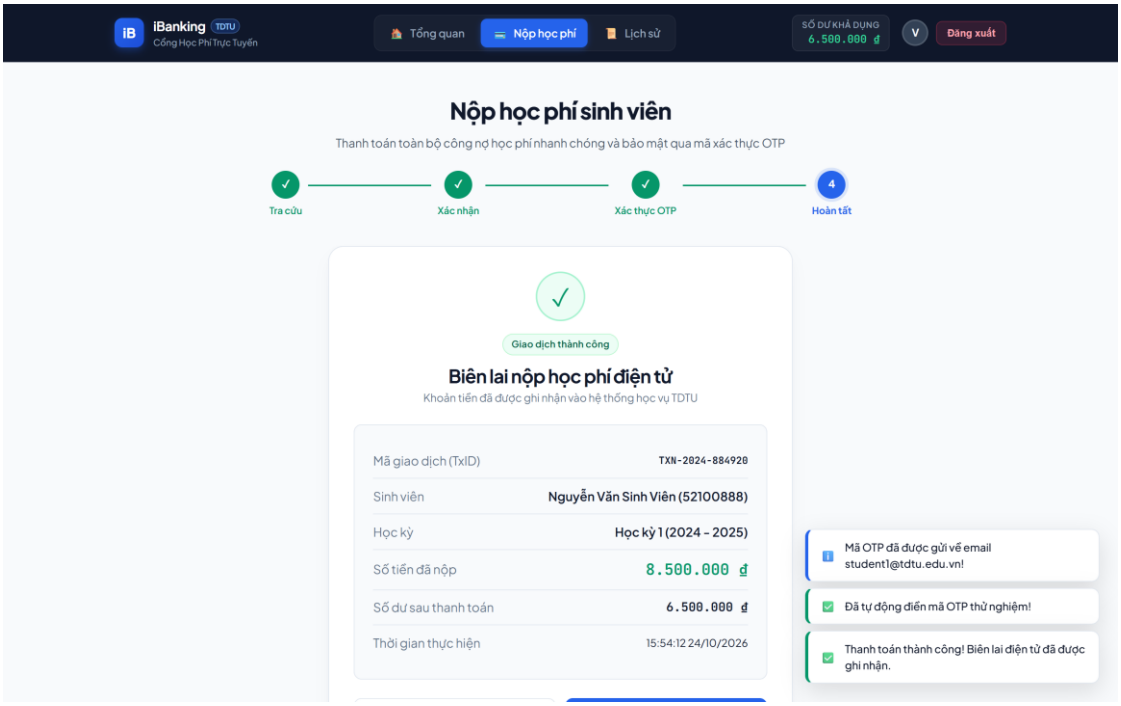
5.1.4 Email thông báo và lịch sử giao dịch

Notification Worker nhận thông điệp qua hàng đợi RabbitMQ và sử dụng giao thức SMTP (Gmail) để chuyển mã OTP cũng như thư xác nhận thanh toán đến địa chỉ

email của người dùng. Sau khi hoàn tất, giao dịch được lưu lại trên màn hình Lịch sử giao dịch.



Hình 5.5: Email chứa mã xác thực OTP gửi qua giao thức SMTP



Hình 5.6: Biên lai thanh toán và màn hình lịch sử giao dịch

5.2 Concurrency Testing

Nhóm sử dụng kịch bản kiểm thử tự động (tệp scripts/test_concurrency_scenarios.py) để kiểm tra tính toàn vẹn dữ liệu khi có nhiều yêu cầu gửi đến đồng thời. Kết quả thực nghiệm được tổng hợp trong Bảng 5.1.

STT	Kịch bản kiểm thử	Số luồng đồng thời	Kết quả thực tế	Đánh giá tính toàn vẹn
1	Một tài khoản gửi hai yêu cầu thanh toán cùng lúc (Double-spending)	2 luồng song song	Giao dịch đầu tiên thành công (200 OK). Giao dịch thứ hai bị từ chối do không đủ số dư (400 Bad Request).	Đạt (Số dư không bị trừ âm)
2	Hai tài khoản cùng thanh toán cho một khoản học phí	2 luồng song song	Giao dịch 1 gạch nợ thành công (PAID). Giao dịch 2 nhận phản hồi lỗi (409 Conflict: Already paid).	Đạt (Học phí chỉ ghi nhận một lần)
3	Gửi lặp lại yêu cầu tạo giao dịch từ giao diện trong thời gian ngắn	5 yêu cầu liên tiếp	Hệ thống lọc các yêu cầu trùng lặp nhờ Idempotency Key, chỉ khởi tạo duy nhất 1 giao dịch.	Đạt (Đảm bảo tính lũy đẳng)

Bảng 5.1: Kết quả kiểm thử tương tranh và tính toàn vẹn dữ liệu

5.3 Saga Rollback & Compensation

Trong mô hình Database-per-service, khi một bước trong tiến trình thanh toán gặp sự cố, hệ thống thực hiện bù trừ theo quy trình:

- Kịch bản kiểm thử: User Service trừ tiền trong tài khoản người dùng thành công, nhưng yêu cầu thanh toán sang Tuition Service bị từ chối vì khoản học phí đã được gạch nợ trước đó (mã lỗi 409 Conflict).
- Xử lý bù trừ: Payment Orchestrator ghi nhận mã lỗi 409 và gửi yêu cầu hoàn tiền (operation: refund) đến User Service để hoàn lại số dư tương ứng.

- Kết quả: Giao dịch được chuyển sang trạng thái REFUNDED, số dư tài khoản người dùng phục hồi nguyên trạng và hệ thống lưu lại nhật ký đối soát.

5.4 Technical Deliverables Evaluation

STT	Yêu cầu kỹ thuật đồ án	Hiện thực trong hệ thống	Kết quả
1	Kiến trúc Microservices độc lập	6 dịch vụ (Auth, User, Tuition, Payment, OTP, Notification) cùng 1 Gateway	Đạt
2	Database-per-Service	5 cơ sở dữ liệu PostgreSQL riêng biệt, không sử dụng ràng buộc khóa ngoại chéo giữa các dịch vụ	Đạt
3	Giao dịch phân tán (Distributed Transaction)	Điều phối qua Payment Orchestrator kèm cơ chế bù trừ (refund) tự động khi có bước thất bại	Đạt
4	Kiểm soát tương tranh (Concurrency Control)	Khóa bi quan (SELECT ... FOR UPDATE) tại tầng cơ sở dữ liệu để ngăn ngừa xung đột	Đạt
5	Giao tiếp bất đồng bộ qua Message Queue	Hàng đợi thông điệp RabbitMQ (otp_email, payment_success) và gửi thông báo qua Gmail SMTP	Đạt
6	Container hóa và triển khai	Đóng gói toàn bộ dịch vụ bằng Docker Compose kèm kịch bản chạy tự động (start.bat / start.sh)	Đạt
7	Giao diện người dùng	Giao diện React.js với quy trình 4 bước, cập nhật số dư và hiển thị biên lai điện tử	Đạt

Bảng 5.2: Bảng đối soát mức độ hoàn thành các tiêu chí kỹ thuật đồ án

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Đề tài đã hoàn thành việc xây dựng và kiểm thử hệ thống thanh toán học phí trực tuyến iBanking theo kiến trúc Microservices đáp ứng đầy đủ các mục tiêu đề ra. Hệ thống giải quyết hiệu quả bài toán quản lý giao dịch phân tán thông qua Saga Orchestration kèm cơ chế hoàn tiền tự động khi xảy ra lỗi. Vấn đề tranh chấp dữ liệu (Race Condition) và thanh toán trùng lặp được loại bỏ nhờ áp dụng khóa bi quan (SELECT ... FOR UPDATE) tại cơ sở dữ liệu và Idempotency Key từ phía ứng dụng. Đồng thời, luồng xác thực hai lớp (2FA) sử dụng mã OTP qua hàng đợi RabbitMQ và SMTP Gmail bảo đảm an toàn cho các giao dịch tài chính.

6.2 Hướng phát triển

• Tích hợp Redis Cache: Triển khai bộ nhớ đệm phân tán để giảm tải truy vấn cho Tuition Service và User Service trong các đợt cao điểm tra cứu.

• Quản lý nhật ký tập trung (ELK Stack): Xây dựng hệ thống thu thập và phân tích log tập trung thông qua Elasticsearch, Logstash và Kibana nhằm theo dõi luồng giao dịch phân tán.

• Giám sát hệ thống thời gian thực (Prometheus & Grafana): Thu thập số liệu hiệu năng (Request latency, Error rate, Queue size) để cảnh báo sớm sự cố của từng dịch vụ.

TÀI LIỆU THAM KHẢO

[1] Sam Newman, Building Microservices: Designing Fine-Grained Systems, 2nd Edition, O'Reilly Media, 2021.

[2] Chris Richardson, Microservices Patterns: With Examples in Java, Chapter 4: 'Managing transactions with sagas', Manning Publications, 2018.

[3] Martin Fowler, 'Saga Pattern: Distributed Transactions in Microservices', martinfowler.com, 2017.

[4] FastAPI Documentation, 'Asynchronous Web Framework for Python', truy cập tại: https://fastapi.tiangolo.com/

[5] RabbitMQ Documentation, 'Reliable Asynchronous Messaging with AMQP 0-9-1', truy cập tại: https://www.rabbitmq.com/

[6] Giảng viên Dương Hữu Phúc, 'SOA Resources & Microservices Architecture Guidelines', Khoa Công nghệ Thông tin, Trường Đại học Tôn Đức Thắng, truy cập tại: https://soa.duonghuuphuc.com/resources

[7] Nhóm tác giả, 'Mã nguồn Hệ thống thanh toán học phí iBanking TDTU', GitHub Repository, truy cập tại: https://github.com/2006phu/SOA-ibanking